Online Judge

Problem:

Design an Online Judge platform where users can access problems, submit their code, and get a verdict.

Solution:

For version 1, develop a simple platform with problem list page, problem statement page, and an online compiler supporting multiple languages.

For version 2, integrate an AI assistant on problem page to provide hints (level 1, 2, 3) based on user code and thinking, generate a short report on wrong submission to help identify reason for failure, and suggest next problem based on previous submissions.

Features:

- Authentication
- Dashboard/ Problem lists
- Problem statement and code submission
- Leaderboard
- Profile
- AI guidance

HLD:

1. **Database Design-**

- **User**
  UserID: CharField
  Email: CharField
  Password: CharField
  FullName: CharField
  Score: Numeric
- **Problem**
  ProblemID: CharField
  Title: CharField
  Statement: CharField
  Difficulty: CharField

Topics: Array of Char/String

- **TestCase**
  ProblemID: CharField (Foreign Key)
  TestCaseID: CharField
  Input: CharField
  Output: CharField
- **Submission**
  SubmissionNo: Numeric

  ProblemID: CharField (Foreign Key)

  UserID: CharField (Foreign Key)

  Verdict: CharField

  Time: Auto Date Time

2. **Frontend – Using ReactJS**

- **LoginPage**
  Input login credentials email and password, verify and route to dashboard.
  Direct to register page for new users.
- **RegisterPage**
  Take user data: Fullname, email, password
  Generate a unique user id, register user on database and login.
  Redirect to dashboard.
- **Dashboard/ProblemList**

  Show all the available problems in a table form, display problem id, title, status (solved/unsolved/verdict), and difficulty

  Redirect to problem page on item click.

- **ProblemPage**
  The actual page showing detailed problem statement, example test cases, attached text editor for code, option to select language, local code file, and run & submit buttons. Show verdict on submit. Contain an AI bot to assist, help debug and suggest optimizations.
- **SubmissionsPage**
  This page shows all the submissions made by user in table format.
- **Leaderboard**

Show all top performers by checking score based on no. of problems and accuracy.

- **Profile**
  Show user data, score, and progress chart
- **Compiler**
  An in-built compiler to run custom codes. Contain a text editor, input section and show output on run.

3. **Backend Logic**

- **Authentication**
  Define an API to GET user from database
  Validate user credentials
  POST user data for new user and hash the password
  Create JWT token and login
- **Problem List**
  Define an API endpoint in Express.js that handles a GET request to fetch all problems from the database (MongoDB) and return them to the frontend.
- **Specific Problem**
  Define an API to handle GET request to fetch complete problem details from the database.
- **Submission**
  Define an API to handle POST request.
  Retrieve the test cases for the problem.
  Evaluate the code using compiler backend (Docker) and get the output.
  Compare the output with expected output and return verdict to the frontend.
- **Submission List**
  Handle GET request to return all past submissions to the frontend.

4. **Compilation: Docker**
   Create dedicated containers for different languages with limited time and memory to prevent malicious code from crashing the system
   Take submitted code and run using predefined commands
   Accept inputs from test case and generate output
   Return error if time limit exceeds, memory overflow or runtime error as verdict

Compare the output with test case expected output on backend and return verdict

5. **AI integration**
   Use Gemini API/ AI Studio to get personalized guidance, submission report with possible optimization, complexities and problem recommendation.
   Show verified possible issues by recompiling correct code. Correct it 2-3 times until passed to avoid wrong hints. Generate response on user requirement to avoid unnecessary API calls.